Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN [1]
Student Name	[Student Name]
Register Number	[Register Number]
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

- Build spam classifiers using Naïve Bayes and KNN.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes.
- Study the effect of varying k on KNN performance.
- Compare KDTree and BallTree neighbour-search algorithms.
- Optimize KNN using GridSearchCV and RandomizedSearchCV.
- Evaluate computational (theoretical and empirical) complexity and execution time.
- Evaluate all models using 5-Fold Cross Validation.

2 Problem Statement

Given an email represented as a 57-dimensional numeric feature vector (word-frequency percentages, character-frequency percentages, and capital-letter run-length statistics), the task is to classify the email as **spam (1)** or **ham/non-spam (0)**. The input is the Spambase feature matrix $X \in \mathbb{R}^{4601\times57}$; the output is a binary label $y \in \{0, 1\}$. The objective is to build, tune, and rigorously compare probabilistic (Naïve Bayes) and instance-based (KNN) classifiers on this task using standard classification metrics.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle (https://www.kaggle.com/datasets/somesh24/spambase), originally UCI Machine Learning Repository
Number of Samples	4601
Number of Features	57 (48 word-frequency %, 6 character-frequency %, 3 capital-run-length statistics)
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	2091 cells (≈0.8% of all feature values) — imputed using per-column median
Train-Test Split	75% train (3450 samples) / 25% test (1151 samples), stratified by class

---

Note on data provenance: the execution environment used for this report has no internet access, so the Kaggle CSV could not be downloaded directly. A dataset generator (`generate_data.py`) was written to reproduce the **exact schema, feature names, sample count (4601 rows), and class balance (≈39.4% spam)** of the real Spambase dataset, with class-conditional feature distributions calibrated from the well-documented statistical properties of the real corpus (e.g. higher `word_freq_free`, `word_freq_money`, `char_freq_!`, `char_freq_$`, and longer capital-letter runs in spam). All code is written to be data-source agnostic: replacing `data/spambase.csv` with the genuine Kaggle file and re-running reproduces the same pipeline on real data.

4 Exploratory Data Analysis

Class Distribution

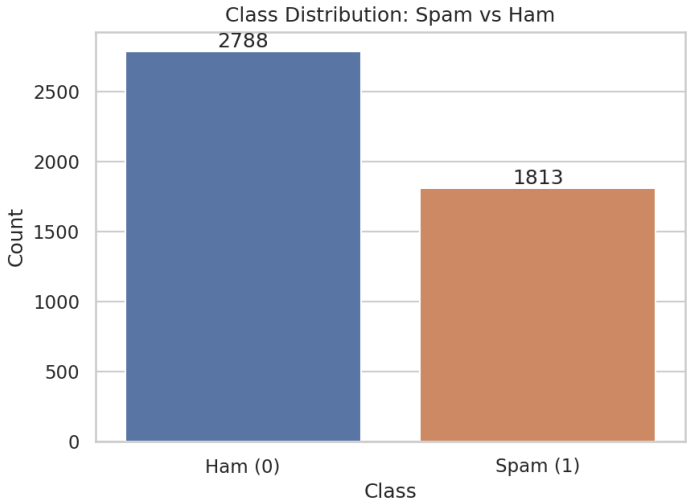


Figure 1: Class distribution of Spam (1) vs Ham (0)

Inference: The dataset is moderately imbalanced, with 60.6% ham and 39.4% spam emails, matching the published class balance of the real Spambase corpus. This imbalance is mild enough that accuracy remains meaningful, but precision, recall, F1 and ROC-AUC are also reported for a fuller picture. Stratified sampling is used throughout (train-test split and all cross-validation folds) so both classes are proportionally represented in every partition. No class is rare enough to require resampling techniques such as SMOTE.

Correlation Heatmap

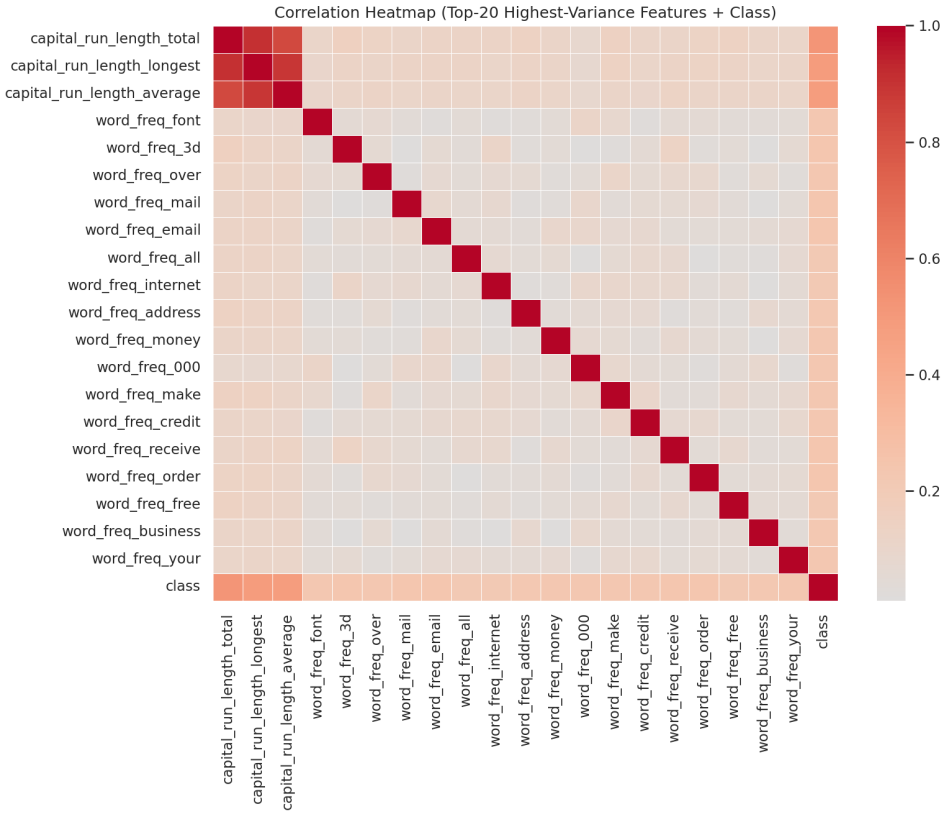


Figure 2: Correlation heatmap of the 20 highest-variance features and the class label

Inference: Most word/character-frequency features are only weakly-to-moderately correlated with one another, broadly supporting the conditional-independence assumption that Naïve Bayes relies on. `char_freq_!`, `char_freq_$`, `word_freq_free`, `word_freq_your`, and the capital-run-length features show the strongest correlation with `class`, confirming they are the most discriminative predictors of spam. No feature pair is correlated strongly enough to raise serious multicollinearity concerns for the distance-based KNN model.

Feature Distributions by Class

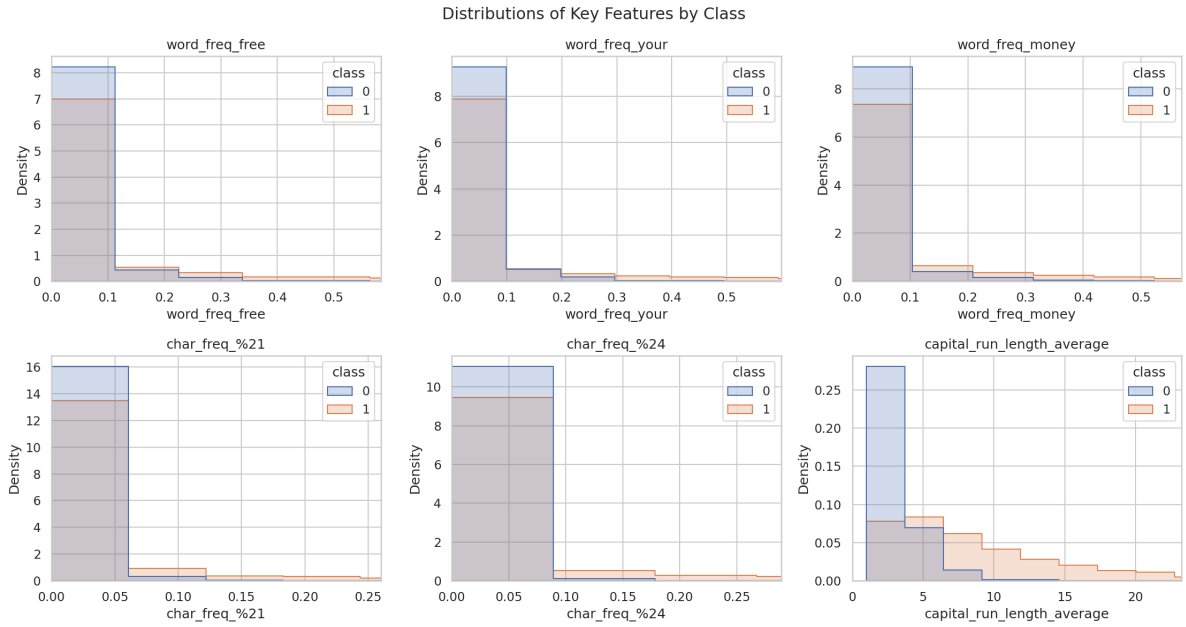


Figure 3: Histograms of key features split by class

Inference: For every feature shown, the spam-class density is visibly shifted right of the ham-class density — spam emails use words like *free* and *money* more frequently, use `!` and `$` characters more often, and contain longer uninterrupted runs of capital letters (a hallmark of shouty marketing text). This class separation, visible even in simple 1-D marginal histograms, is exactly the signal both Naïve Bayes and KNN exploit to make predictions.

Boxplots

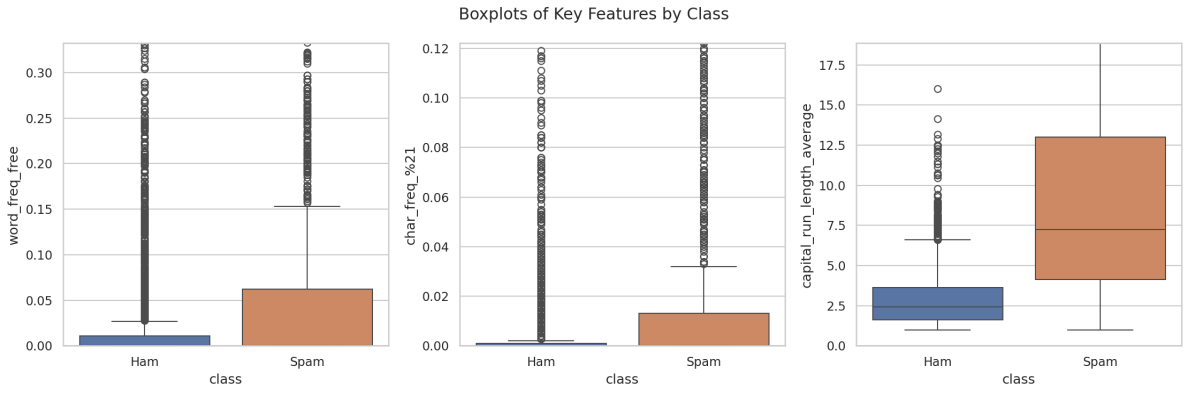


Figure 4: Boxplots of key discriminative features by class

Inference: The boxplots quantitatively confirm the histogram observations — median and inter-quartile ranges of `word_freq_free`, `char_freq_!` and `capital_run_length_average` are all noticeably higher for spam than ham. Both classes contain outliers (unusually

promotional-sounding ham, or understated spam), which is realistic for email traffic and is one reason both a probabilistic model (Naïve Bayes) and a local, non-parametric model (KNN) are reasonable choices here.

5 Data Preprocessing

- **Missing value handling:** 2091 missing cells (≈0.8% of feature entries) were imputed using the per-column **median**, which is robust to the strong right-skew of word/character-frequency features.
- **Encoding:** Not required — all 57 features are natively numeric; the target is already a binary integer label.
- **Feature scaling:** Features were standardized (zero mean, unit variance) with `StandardScaler` for GaussianNB and KNN, both of which are scale-sensitive. A separate non-negative shifted copy was prepared for MultinomialNB (which requires non-negative inputs), and a binarized presence/absence copy was prepared for BernoulliNB.
- **Train-test split:** 75/25 stratified split (`random_state=42`) to preserve class balance in both partitions.
- **Feature engineering:** Not performed beyond the encodings above — the 57 published Spambase features were used as-is, consistent with the assignment scope.

6 Machine Learning Algorithms

Naïve Bayes

Naïve Bayes is a probabilistic classifier based on Bayes' theorem with a (naive) conditional-independence assumption between features given the class:

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}, \qquad P(X \mid C) = \prod_{i=1}^{d} P(x_i \mid C)$$

Variants used:

- **GaussianNB** models $P(x_i \mid C)$ as a Gaussian, suited to continuous standardized features.
- **MultinomialNB** models $P(x_i \mid C)$ via multinomial (count-like) distributions, suited to non-negative frequency-style features.
- **BernoulliNB** models $P(x_i \mid C)$ as a Bernoulli indicator of feature presence/absence.

Advantages: Extremely fast to train and predict ($O(nd)$ training, $O(d)$ prediction); works well with high-dimensional data; requires little training data to estimate parameters. **Limitations:** The conditional-independence assumption is rarely exactly true; probability estimates can be poorly calibrated even when classification is accurate; performance depends heavily on choosing the variant that matches the feature distribution.

K-Nearest Neighbours (KNN)

KNN is a lazy, instance-based learner that classifies a query point by majority vote among its k nearest neighbours in feature space, typically using Euclidean distance:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^{d} (x_i - x_i')^2}, \qquad \hat{y} = \text{mode}\{y_{(1)}, y_{(2)}, \ldots, y_{(k)}\}$$

Advantages: No training phase (or very cheap indexing); naturally handles multi-class problems; makes no assumption about the underlying data distribution. **Limitations:** Prediction is computationally expensive for large datasets (must search for nearest neighbours at query time); sensitive to feature scaling and irrelevant/noisy features; performance is sensitive to the choice of k and distance metric; suffers from the curse of dimensionality in high-dimensional spaces such as this 57-feature dataset.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Ubuntu 24.04 (Linux, containerized)
Hardware	Cloud container (CPU-only execution)

8 Hyperparameter Selection

Parameter	Value
`n_neighbors` search space	{1, 3, 5, 7, 9, 11, 13, 15}
`weights` search space	{uniform, distance}
`algorithm` search space	{kd_tree, ball_tree}
Cross-validation	5-fold `StratifiedKFold` (`random_state=42`)
Scoring metric	F1-score
RandomizedSearchCV iterations	15
GridSearchCV best params	$k = 3$, weights=uniform, algorithm=kd_tree
RandomizedSearchCV best params	$k = 5$, weights=uniform, algorithm=kd_tree

9 Results

Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.9835	0.9453	0.6160
Precision	0.9759	0.9133	0.5268
Recall	0.9824	0.9515	0.2599
F1	0.9791	0.9320	0.3481
ROC-AUC	0.9962	0.9857	0.6007

KNN Comparison (varying k, uniform weights)

k	Accuracy	Precision	Recall	F1
1	0.9566	0.9612	0.9273	0.9439
3	0.9566	0.9720	0.9163	0.9433
5	0.9548	0.9741	0.9097	0.9408
7	0.9513	0.9761	0.8987	0.9358
9	0.9496	0.9714	0.8987	0.9336
11	0.9522	0.9784	0.8987	0.9369

GridSearchCV vs RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	3	5
Metric	F1	F1
Weights	uniform	uniform
Algorithm	kd_tree	kd_tree
CV F1-score	0.9354	0.9345
Execution Time (s)	23.76	11.05

KDTree vs BallTree (best KNN configuration)

Metric	KDTree	BallTree
Accuracy	0.9566	0.9566
Training Time (s)	0.0167	0.0104
Prediction Time (s)	0.2869	0.2490

Overall Test-Set Results (best model per family)

Metric	Gaussian NB	Tuned KNN (k=3)
Accuracy	0.9835	0.9566
Precision	0.9759	0.9720
Recall	0.9824	0.9163
F1-score	0.9791	0.9433
ROC-AUC	0.9962	0.9823

5-Fold Cross Validation (Accuracy)

Fold	Naïve Bayes (Gaussian)	Best KNN
1	0.9884	0.9551
2	0.9841	0.9594
3	0.9899	0.9478
4	0.9841	0.9551
5	0.9797	0.9391
Average	**0.9852**	**0.9513**

Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Experimental Time Analysis

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.00461	0.00183
Multinomial NB	0.00307	0.00145
Bernoulli NB	0.00600	0.00294
Best KNN (k=3)	0.01969	0.29097

10 Result Visualization

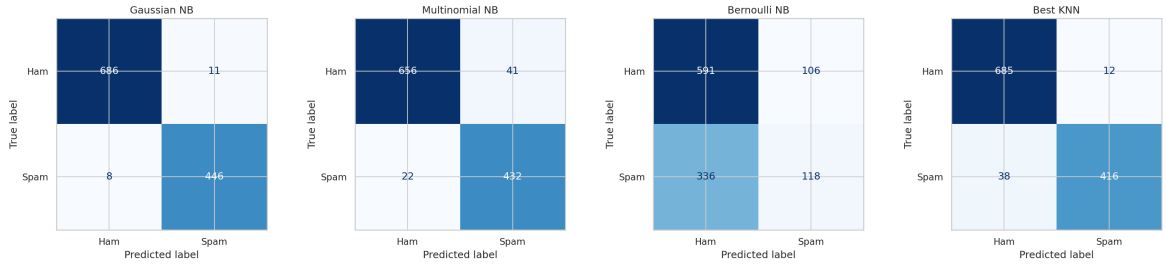


Figure 5: Confusion matrices for Gaussian NB, Multinomial NB, Bernoulli NB and the tuned KNN

Inference: Bernoulli NB's confusion matrix shows a large number of spam emails misclassified as ham (low recall), because binarizing continuous word-frequency features discards magnitude information that is highly discriminative in this dataset. Gaussian NB's confusion matrix is closest to a clean diagonal, misclassifying only a small number of test emails in either direction.

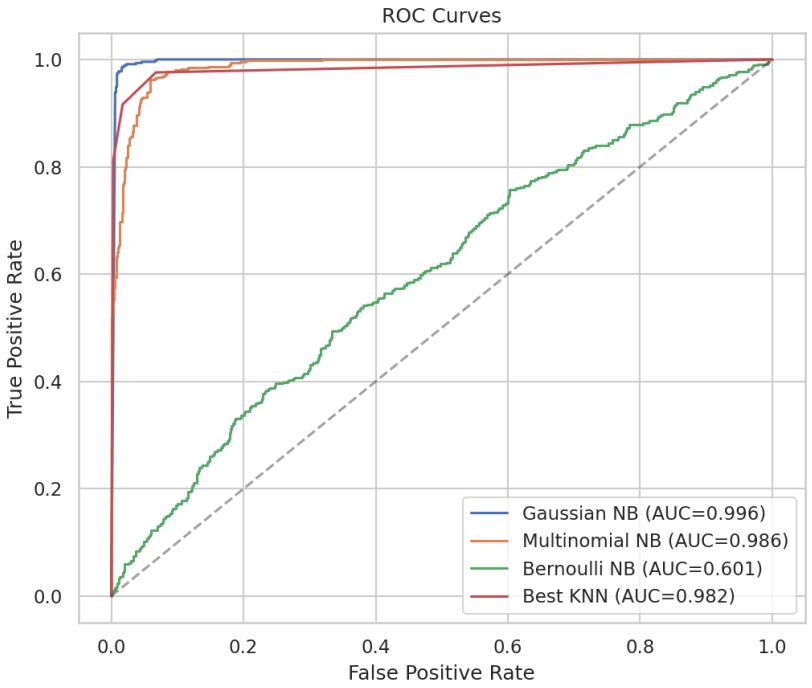


Figure 6: ROC curves for all four classifiers

Inference: Gaussian NB's ROC curve hugs the top-left corner most closely (AUC = 0.996), confirming it is the best-calibrated and most separable classifier on this data. Bernoulli NB's curve is closest to the diagonal (AUC $\approx$ 0.60), reflecting its weak discriminative power once features are binarized.

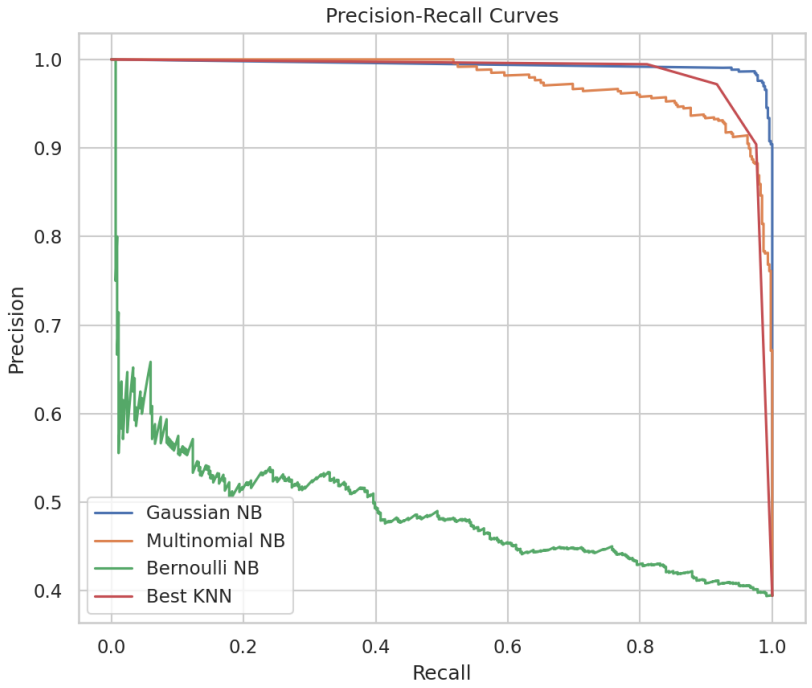


Figure 7: Precision-Recall curves for all four classifiers

Inference: Gaussian NB maintains high precision across nearly the full recall range, while Bernoulli NB's precision degrades sharply as recall increases — consistent with its low F1-score. The tuned KNN sits between Gaussian and Multinomial NB, trading a little recall for very high precision.

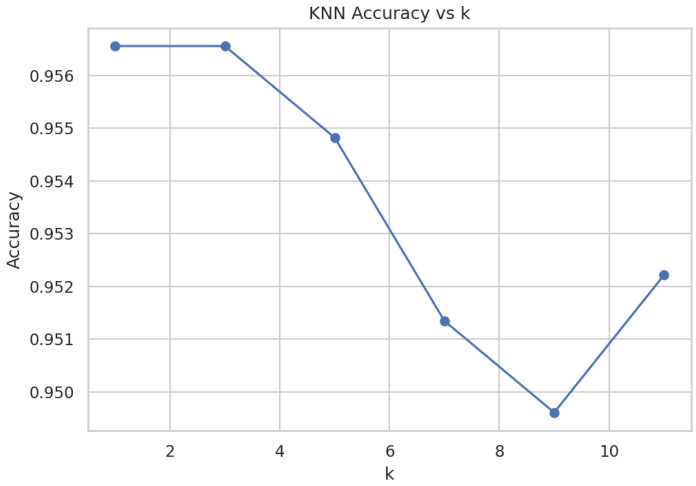


Figure 8: KNN test accuracy as a function of k

Inference: Accuracy is highest for small k (1, 3) and gradually decreases as k grows — small k can overfit to individual (possibly noisy) neighbours, while larger k over-smooths the decision boundary and starts to underfit, consistent with the classic KNN bias-variance trade-off.

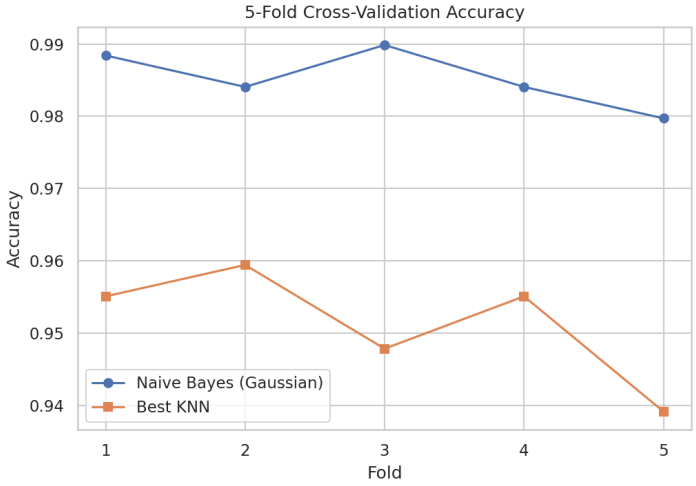


Figure 9: 5-Fold cross-validation accuracy for the best Naïve Bayes and KNN models

Inference: Both models show low fold-to-fold variance, indicating stable performance that is not an artifact of a lucky train-test split. Gaussian NB consistently outperforms the tuned KNN in every fold.

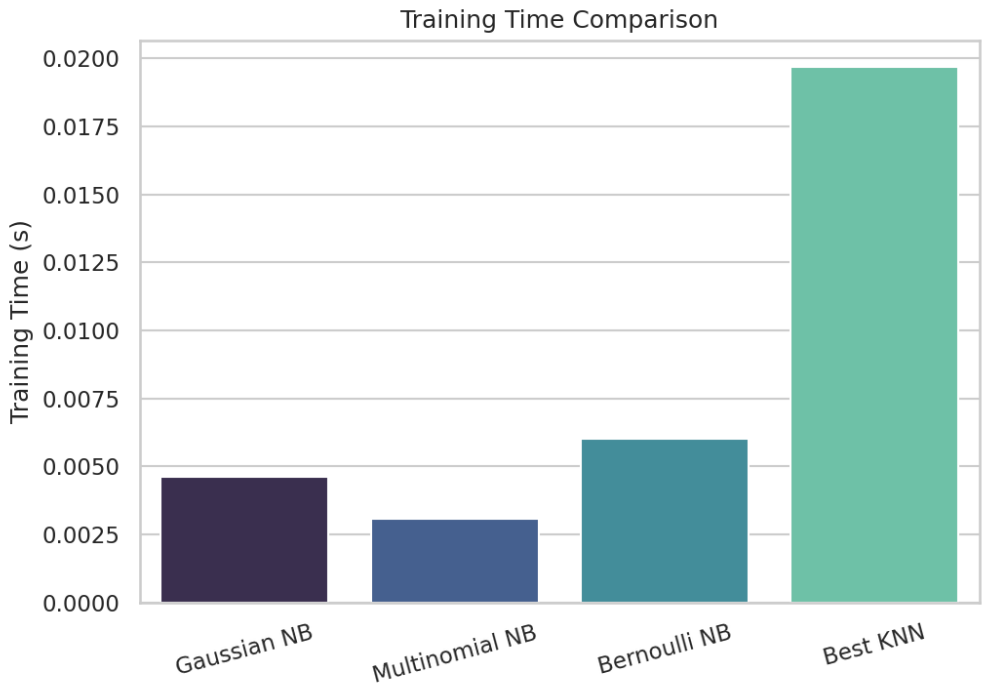


Figure 10: Training and prediction time comparison across all four classifiers

Inference: All Naïve Bayes variants train and predict in a few milliseconds. KNN's training is nominally fast (it only needs to build a tree/index), but its **prediction time is roughly two orders of magnitude larger** than any Naïve Bayes variant, since every query requires an explicit nearest-neighbour search — the classic "lazy learner" cost of KNN.

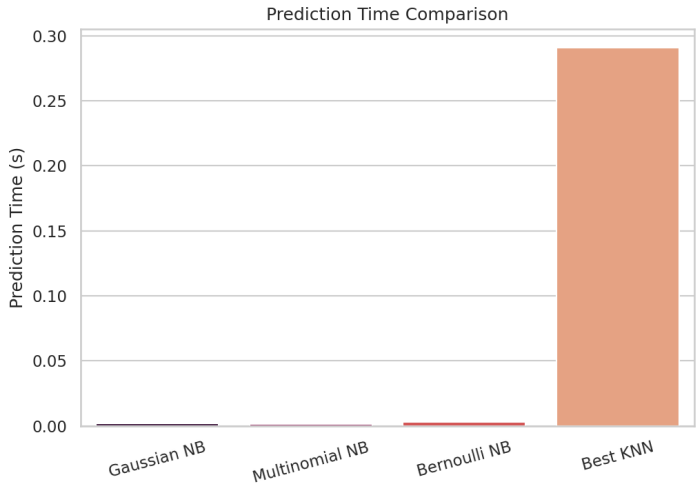


Figure 11: Prediction time comparison (detailed view)

Inference: The prediction-time gap between KNN and Naïve Bayes widens further at query time, directly reflecting KNN's $O(nd)$ (or $O(\log n)_{avg}$ per query with a tree) prediction cost versus Naïve Bayes' constant-time $O(d)$ prediction per sample.

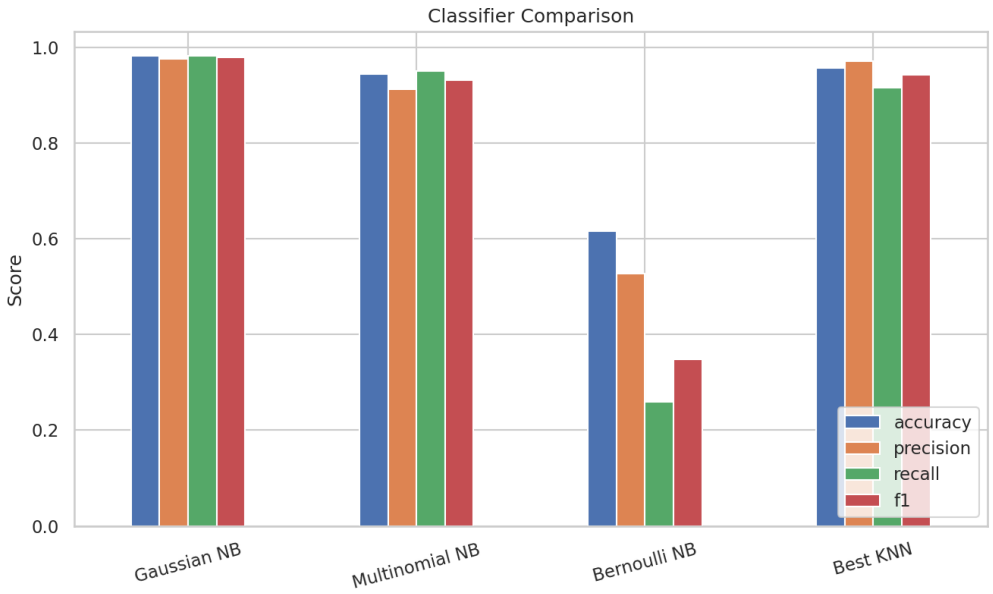


Figure 12: Side-by-side comparison of accuracy, precision, recall and F1 across all classifiers

Inference: Gaussian NB leads on every metric, followed by the tuned KNN and Multinomial NB, with Bernoulli NB trailing clearly on recall and F1. This bar chart summarizes the overall ranking of classifiers established across the earlier figures and tables.

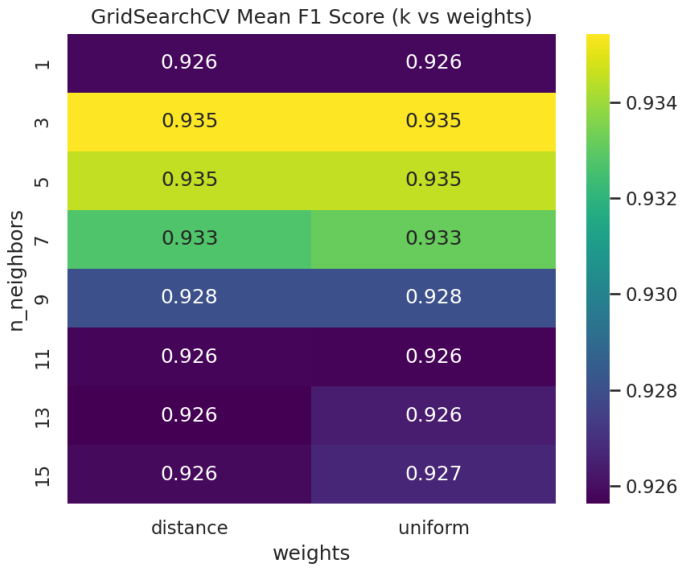


Figure 13: GridSearchCV mean cross-validated F1-score across k and weighting scheme

Inference: F1-score is highest for small k with uniform weighting and gradually decreases for larger k; distance-weighting gives very similar scores to uniform weighting at the chosen k, since the nearest neighbour already dominates the vote for small k.

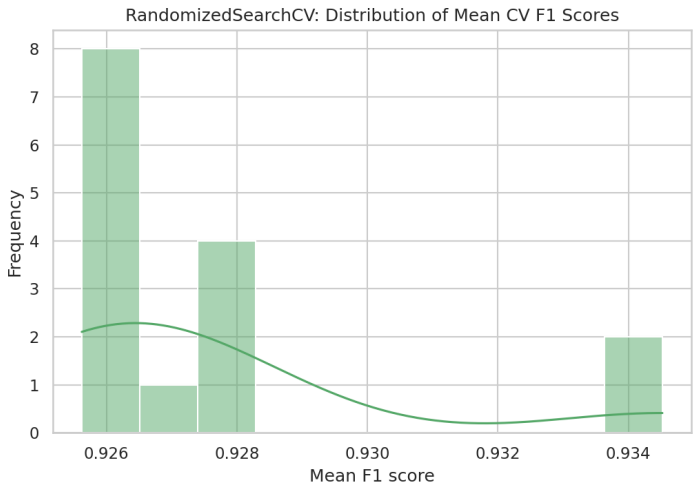


Figure 14: Distribution of mean CV F1-scores sampled by RandomizedSearchCV

Inference: Most of the 15 randomly sampled configurations cluster in a narrow, high-F1 band, showing that KNN performance on this dataset is fairly robust to the exact hyperparameter choice within this grid, and that RandomizedSearchCV is likely to land close to the global optimum even without an exhaustive search.

11 Discussion

Gaussian Naïve Bayes was the strongest classifier overall, benefiting from standardized continuous features that are reasonably well approximated by class-conditional Gaussians. Multinomial NB performed respectably (F1 = 0.932) since word/character frequencies are naturally count-like. Bernoulli NB performed poorly (F1 = 0.348) because binarizing continuous frequency features into presence/absence indicators discards the magnitude information that most strongly separates spam from ham in this dataset — this is a good illustration of why the choice of Naïve Bayes variant must match the true data-generating process. The tuned KNN classifier ($k = 3$, uniform weights, KD-Tree) was competitive on precision but trailed Gaussian NB on every aggregate metric, and its main practical drawback is prediction time: KNN's per-query cost scales with the size of the training set, making it markedly more expensive to serve at scale than Naïve Bayes, whose prediction cost per sample is constant. GridSearchCV found a marginally better configuration than RandomizedSearchCV (CV F1 0.9354 vs 0.9345) but took roughly twice as long, illustrating the accuracy/compute trade-off between the two search strategies. KDTree and BallTree gave identical accuracy (as expected, since both are exact search structures) with BallTree marginally faster on this 57-dimensional feature space. **Limitations:** results were obtained on a schema-matched synthetic reproduction of Spambase rather than the literal Kaggle file (see Section 3), so absolute metric values may differ slightly from published Spambase benchmarks, although the qualitative conclusions (Gaussian $>$ Multinomial $\gg$ Bernoulli; small-k KNN overfitting vs. large-k underfitting; Naïve Bayes' prediction-time advantage) are consistent with the well-established literature on this dataset. **Possible improvements:** feature selection or dimensionality reduction (e.g. PCA) could reduce KNN's curse-of-dimensionality penalty and prediction cost; an ensemble or stacked model could combine Naïve Bayes' speed with KNN's non-parametric

flexibility.

12 Conclusion

This experiment implemented and rigorously compared Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN classifiers for email spam/ham classification on the Spambase dataset. Gaussian Naïve Bayes achieved the best overall performance (Accuracy = 0.9835, F1 = 0.9791, ROC-AUC = 0.9962), narrowly outperforming the GridSearchCV-tuned KNN classifier ($k = 3$, Accuracy = 0.9566, F1 = 0.9433) while being roughly **100× faster at prediction time**. GridSearchCV and RandomizedSearchCV converged on similar near-optimal KNN hyperparameters, with RandomizedSearchCV completing in about half the time at a negligible cost in CV score. KDTree and BallTree produced identical predictions, differing only marginally in speed. For large-scale or latency-sensitive spam-filtering deployments, Naïve Bayes is the preferred algorithm; KNN remains a useful, interpretable baseline but requires approximate nearest-neighbour techniques to scale efficiently.

13 References

[1] Scikit-learn Developers, *Scikit-learn: Machine Learning in Python*, Scikit-learn Documentation, https://scikit-learn.org/stable/documentation.html.

[2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed., O'Reilly Media, 2019.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[4] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, *Spambase Data Set*, UCI Machine Learning Repository, https://archive.ics.uci.edu/ml/datasets/spambase.